

Tarefa 3 — Série de Leibniz: sequencial e com threads

Objetivo

Implementar a série de Leibniz para aproximar π , observar como a precisão evolui com o aumento de iterações, e avaliar o ganho de tempo ao distribuir o trabalho entre múltiplas threads com OpenMP.

O que é a série de Leibniz

A série de Leibniz é uma soma infinita que converge para $\pi/4$:

$$\pi/4 = 1 - 1/3 + 1/5 - 1/7 + 1/9 - \dots$$

Quanto mais termos somados, mais o resultado se aproxima de π . O problema é que a convergência é muito lenta — cada 10 vezes mais termos acrescenta apenas uma casa decimal correta.

Versão sequencial

O programa soma os termos um por um, do primeiro até o N-ésimo, e mede o tempo total.

Resultados

Iterações	Tempo (s)	π aproximado	Erro
10	5.0×10^{-7}	3.041839618929402	9.975×10^{-2}
100	1.9×10^{-7}	3.131592903558552	1.000×10^{-2}
1.000	1.4×10^{-6}	3.140592653839792	1.000×10^{-3}
10.000	1.5×10^{-5}	3.141492653590043	1.000×10^{-4}
100.000	1.5×10^{-4}	3.141582653589793	1.000×10^{-5}
1.000.000	1.4×10^{-3}	3.141591653589793	1.000×10^{-6}
10.000.000	1.5×10^{-2}	3.141592553589793	1.000×10^{-7}
100.000.000	1.4×10^{-1}	3.141592643589794	1.000×10^{-8}
1.000.000.000	1.41	3.141592652589795	1.000×10^{-9}
5.000.000.000	7.33	3.141592653389802	2.000×10^{-10}

Análise

O padrão é claro: **cada vez que o número de termos multiplica por 10, o erro divide por 10** — ou seja, ganha-se uma casa decimal correta. Isso é chamado de convergência linear.

Comparando os resultados dígito a dígito com π real:

```

π real:      3.14159265358979323...
10 iter:     3.04183961892940221... → 1 casa correta
100 iter:    3.13159290355855276... → 2 casas corretas
1000 iter:   3.14059265383979292... → 3 casas corretas
...
5B iter:     3.14159265338980221... → 9 casas corretas

```

Para chegar a 9 casas decimais corretas foram necessárias **5 bilhões de iterações** e mais de **7 segundos** de execução. E cada casa decimal a mais exige 10 vezes mais trabalho.

Com threads (OpenMP)

A versão paralela divide as iterações entre múltiplas threads. Cada thread soma uma fatia do vetor de forma independente e os resultados são combinados no final. O número de iterações é o mesmo — apenas o trabalho é distribuído.

Resultados por número de threads

100 milhões de iterações

Threads	Tempo (s)	Speedup
1	0.138	1.0×
2	0.116	1.2×
4	0.050	2.8×
8	0.044	3.1×

1 bilhão de iterações

Threads	Tempo (s)	Speedup
1	1.424	1.0×
2	0.829	1.7×
4	0.463	3.1×
8	0.333	4.3×

5 bilhões de iterações

Threads	Tempo (s)	Speedup
1	7.241	1.0×

Threads	Tempo (s)	Speedup
2	4.031	1.8×
4	2.487	2.9×
8	1.643	4.4×

O que muda — e o que não muda

O tempo cai. Com 8 threads e 5 bilhões de iterações, o tempo vai de 7.2s para 1.6s — quase 4.5× mais rápido.

A precisão não muda. O erro permanece o mesmo para qualquer número de threads. Isso é esperado: as threads estão somando exatamente os mesmos termos, apenas em paralelo. Mais threads não significam mais termos — o algoritmo é o mesmo.

Por que o speedup não chega a 8× com 8 threads

Com 8 threads, seria esperado um ganho de até 8× — mas o resultado fica em torno de 4×. Isso acontece por dois motivos:

- **Overhead de coordenação:** ao final, cada thread precisa comunicar seu resultado parcial para que sejam somados. Esse passo é sequencial e consome tempo.
- **Tamanho do trabalho:** para 100 milhões de iterações (que já rodam em 0.14s), o tempo de preparar as threads representa uma fração relevante do total. Por isso o speedup com 100M é menor do que com 5B — o trabalho é pequeno demais para aproveitar bem as threads.

Tamanho	Speedup com 8 threads
100 milhões	3.1×
1 bilhão	4.3×
5 bilhões	4.4×

Quanto mais pesado o trabalho, melhor o aproveitamento das threads.

Conclusão

A série de Leibniz é um algoritmo de convergência lenta: cada casa decimal extra custa 10 vezes mais iterações. Com 5 bilhões de termos e 7 segundos de execução, o resultado ainda fica longe da precisão máxima que o computador consegue representar.

Paralelizar com OpenMP reduz o tempo de forma significativa — quase 4.5× com 8 threads no caso mais pesado — mas não altera em nada a precisão: o mesmo número de termos, somados mais rápido, chega ao mesmo resultado.

Código

leibniz_seq.c

```
#include <stdio.h>
#include <math.h>
#include <omp.h>

// Referencia com casas extras para long double (18-19 digitos significativos)
#define M_PI1 3.14159265358979323846264338327950288L

// Serie de Leibniz:  $\pi/4 = 1 - 1/3 + 1/5 - 1/7 + \dots$ 
// long double: ~18-19 digitos significativos vs ~15-16 do double
void leibniz(long long n) {
    double start = omp_get_wtime();

    long double sum = 0.0L;
    for (long long i = 0; i < n; i++) {
        long double term = 1.0L / (2.0L * i + 1.0L);
        sum += (i % 2 == 0) ? term : -term;
    }

    long double pi = 4.0L * sum;
    double end = omp_get_wtime();

    long double erro = fabs1(pi - M_PI1);
    // output CSV: iteracoes,segundos,pi_aprox,erro
    printf("%lld,%.9f,%.21Lf,%.3Le\n", n, end - start, pi, erro);
}

int main() {
    long long iteracoes[] = {
        10, 100, 1000, 10000, 100000,
        1000000, 10000000, 100000000,
        1000000000LL, 5000000000LL
    };
    int n = sizeof(iteracoes) / sizeof(iteracoes[0]);

    printf("# M_PI (referencia): %.21Lf\n", M_PI1);
    printf("iteracoes,segundos,pi_aprox,erro\n");
    for (int i = 0; i < n; i++)
        leibniz(iteracoes[i]);

    return 0;
}
```

leibniz_omp.c

```
#include <stdio.h>
#include <math.h>
#include <omp.h>

#define M_PI1 3.14159265358979323846264338327950288L

// Serie de Leibniz paralelizada com OpenMP
// long double: mais precisao; reduction garante soma correta entre threads
void leibniz_omp(long long n, int num_threads) {
    double start = omp_get_wtime();

    long double sum = 0.0L;

    #pragma omp parallel for reduction(+:sum) num_threads(num_threads)
    for (long long i = 0; i < n; i++) {
        long double term = 1.0L / (2.0L * i + 1.0L);
        sum += (i % 2 == 0) ? term : -term;
    }

    long double pi = 4.0L * sum;
    double end = omp_get_wtime();

    long double erro = fabs1(pi - M_PI1);
    // output CSV: iteracoes,threads,segundos,pi_aprox,erro
    printf("%lld,%d,%.9f,%.21Lf,%.3Le\n", n, num_threads, end - start, pi, erro);
}

int main() {
    long long iteracoes[] = {
        100000000LL, 1000000000LL, 5000000000LL
    };
    int threads[] = {1, 2, 4, 8};

    int ni = sizeof(iteracoes) / sizeof(iteracoes[0]);
    int nt = sizeof(threads) / sizeof(threads[0]);

    printf("# M_PI (referencia): %.21Lf\n", M_PI1);
    printf("iteracoes,threads,segundos,pi_aprox,erro\n");
    for (int i = 0; i < ni; i++)
        for (int j = 0; j < nt; j++)
            leibniz_omp(iteracoes[i], threads[j]);

    return 0;
}
```

Compilação e execução

```
gcc -O2 -fopenmp leibniz_seq.c -o leibniz_seq -lm
gcc -O2 -fopenmp leibniz_omp.c -o leibniz_omp -lm

./leibniz_seq
./leibniz_omp
```